

kacchiOS

A Minimal Bare-Metal x86 Operating System

Project Report
CSE 3202: Operating System Sessional
Department of Computer Science & Engineering
Group: OS Project Group B4

Name	Roll No.	GitHub	Module
Md. Sajedul Islam	2103096	sajidruetcse21	Memory Manager (Core)
Tanmoy Mridha	2103118	Tanmoym047	Memory Manager (Advanced)
Nazifa Fairuz Zuthi	2103111	ZohanaZuthi	Process Manager (Core)
Md. Redwan Bhuiyan Rafio	2103113	rafio1020	Process Manager (Utilities & IPC)
Safius Sifat	2103085	Safius-Sifat	Scheduler
Shah Sanjida Rahman Nitu	2103077	Sanjida-Rahman-Nitu	Documentation & Finalization

Repository:
<https://github.com/OS-Project-Group/kacchiOS>

February 24, 2026

Contents

1	Introduction	3
1.1	Design Goals	3
2	System Architecture Overview	3
2.1	Source Tree	3
2.2	Boot Flow	4
2.3	Module Interaction	5
3	Memory Manager	5
3.1	Memory Layout	5
3.2	Stack Allocator	5
3.3	Heap Allocator (Best-Fit + Coalescing)	6
3.3.1	Allocation – Best-Fit Search	6
3.3.2	Deallocation – Forward Merge & Global Coalesce	6
3.4	Stress Test	7
4	Process Manager	7
4.1	Process Control Block (PCB)	7
4.2	Process States	8
4.3	Process Creation	8
4.4	Ready Queue Implementation	9
4.5	Utility Functions (Issue #4)	9
5	Inter-Process Communication (IPC)	10
5.1	API	10
6	CPU Scheduler	10
6.1	Scheduling Policies	10
6.2	Time Quantum	11
6.3	Context Switch	11
6.4	Aging Anti-Starvation Mechanism (Bonus)	11
6.5	Scheduler Data Flow	12
7	Serial I/O Driver	12
8	Build System	12
9	Testing	13
9.1	Memory Tests	13

9.2	Process Manager Tests (10 tests)	14
9.3	IPC Tests (7 tests)	14
9.4	Scheduler Tests (5 tests)	14
10	Team Contributions	15
11	Challenges and Design Decisions	16
12	Conclusion	16

1 Introduction

kacchiOS is a minimal bare-metal operating system developed for x86 (32-bit) architecture entirely from scratch using the C programming language and x86 Assembly. The project was undertaken as part of the OS Lab course to gain hands-on experience of the core mechanisms that power modern operating systems – memory management, process management, CPU scheduling, and inter-process communication (IPC).

The operating system boots directly via a GRUB-compatible Multiboot header, initialises a 16 KB kernel stack in assembly, clears the BSS segment, and then transfers control to the C `kmain()` function. From that point, the following modules take over in sequence:

1. **Serial I/O** – communication with the host through COM1.
2. **Memory Manager** – static stack allocator and best-fit heap allocator with coalescing.
3. **Process Manager** – PCB-based process lifecycle management with a doubly-linked ready queue.
4. **Scheduler** – priority-based Round-Robin scheduler with configurable time quantum and an aging mechanism.
5. **IPC** – a lightweight message-passing subsystem.
6. **Shell** – a read-eval loop on the serial console.

1.1 Design Goals

- Compile and run on bare x86 hardware or QEMU without any host OS services.
- Keep all components self-contained: no libc, no POSIX, no external dependencies.
- Maintain a modular, well-tested codebase – each module has its own unit tests driven directly from `kmain()`.
- Demonstrate graduate-level OS concepts (context switching, scheduling policies, fragmentation handling, IPC) in a compact code base.

2 System Architecture Overview

2.1 Source Tree

```
kacchiOS/  
  src/  
    boot.S          -- Multiboot entry, BSS clear, stack setup
```

```
ctxsw.S      -- x86 context-switch (EBX/ESI/EDI/EBP/EIP)
kernel.c     -- kmain(), integration tests, shell loop
memory.[ch]  -- Stack allocator + best-fit heap
process.[ch] -- PCB, process table, IPC
scheduler.[ch] -- Priority RR scheduler, aging
serial.[ch]  -- COM1 UART driver
string.[ch]  -- Minimal string utilities
io.h         -- Inline port I/O helpers
types.h      -- Primitive type aliases
link.ld      -- Linker script (1 MB load, BSS symbols)
Makefile
README.md
```

2.2 Boot Flow

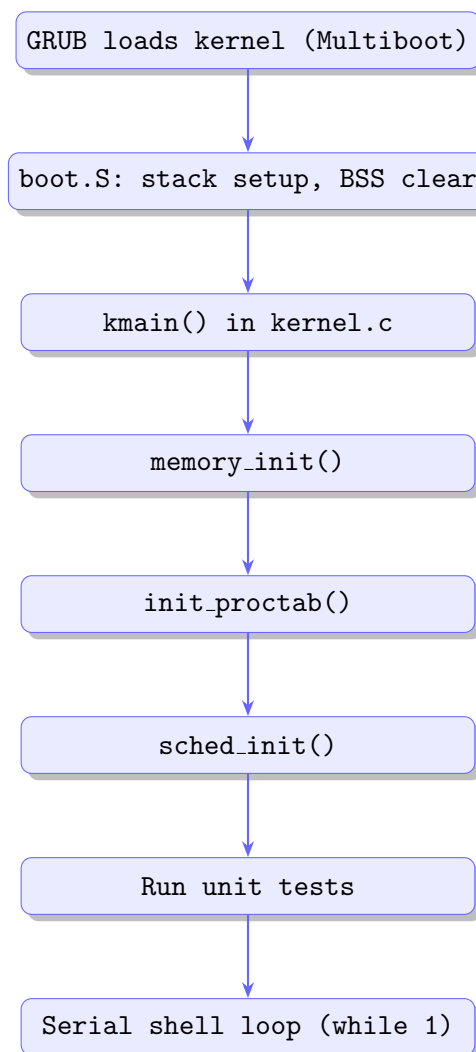


Figure 1: System boot sequence from GRUB to the interactive shell.

2.3 Module Interaction

Caller	Calls into	Purpose
kernel.c	memory.c	Allocate process stacks via <code>heap_alloc()</code>
kernel.c	process.c	Create / terminate processes
kernel.c	scheduler.c	Invoke <code>resched()</code> , <code>yield()</code>
scheduler.c	process.c	Read/write PCB fields, queue ops
scheduler.c	ctxsw.S	Actual register save/restore
process.c	memory.c	Free stack on termination

3 Memory Manager

Primary contributors: Md. Sajedul Islam (Issue #1), Tanmoy Mridha (Issue #2).

3.1 Memory Layout

The memory manager owns two statically allocated regions defined in `memory.c`:

Region	Symbol	Size
Stack region	<code>stack[]</code>	4 096 bytes (4 KB)
Heap region	<code>heap[]</code>	8 192 bytes (8 KB)

These arrays reside in BSS (zero-initialised) and are separate from the 16 KB boot stack configured in `boot.S`.

3.2 Stack Allocator

The stack allocator uses a simple *bump pointer* strategy. A module-level variable `stack_top` tracks the offset of the next free byte.

```

1 void* stack_alloc(size_t size) {
2     if (stack_top + size > STACK_SIZE)
3         return NULL;
4     void* ptr = &stack[stack_top];
5     stack_top += size;
6     return ptr;
7 }
8
9 void stack_free(size_t size) {
10    if (size > stack_top) stack_top = 0;
11    else
12        stack_top -= size;

```

Listing 1: Stack allocation and deallocation (`memory.c`)

Because the bump pointer only allows LIFO-order deallocation, `stack_free` simply decrements the pointer by `size`. This is intentionally simple and mirrors hardware call-stack semantics.

3.3 Heap Allocator (Best-Fit + Coalescing)

The heap is managed through an intrusive linked list of `MemBlock` headers:

```
1 typedef struct MemBlock {
2     size_t size;           // usable bytes after header
3     int    free;           // 1 = free, 0 = allocated
4     struct MemBlock *next;
5 } MemBlock;
```

Listing 2: `MemBlock` structure

3.3.1 Allocation – Best-Fit Search

`heap_alloc(size)` aligns the requested size to a 4-byte boundary and then walks the free-list looking for the *smallest* block that is large enough (best-fit policy). When a suitable block is found and it is large enough to be split (i.e. it has space for a new `MemBlock` header plus at least 4 data bytes), the block is split into an allocated part and a new free remainder.

```
1 size = (size + 3) & ~3;           // align to 4 bytes
2 MemBlock *best_block = NULL;
3 MemBlock *curr = heap_head;
4 while (curr) {
5     if (curr->free && curr->size >= size) {
6         if (!best_block || curr->size < best_block->size)
7             best_block = curr;
8         if (curr->size == size) break;    // exact match
9     }
10    curr = curr->next;
11 }
```

Listing 3: Best-fit heap allocation (excerpt)

3.3.2 Deallocation – Forward Merge & Global Coalesce

`heap_free(ptr)` marks the block as free, then performs a *forward merge* with its immediate successor if it is also free. Afterwards, a *global coalesce* pass walks the entire free list and merges any adjacent free blocks, eliminating external fragmentation.

```
1 block->free = 1;
2 // Forward merge
```

```

3 if (block->next && block->next->free) {
4     block->size += sizeof(MemBlock) + block->next->size;
5     block->next = block->next->next;
6 }
7 // Global coalesce
8 MemBlock *curr = heap_head;
9 while (curr && curr->next) {
10     if (curr->free && curr->next->free) {
11         curr->size += sizeof(MemBlock) + curr->next->size;
12         curr->next = curr->next->next;
13     } else { curr = curr->next; }
14 }

```

Listing 4: Heap free with coalescing (excerpt)

3.4 Stress Test

The `stress_test_memory()` function, called early in `kmain()`, validates the memory subsystem end-to-end:

1. Allocates and deallocates a 100-byte stack block.
2. Allocates three separate 512-byte heap blocks.
3. Frees all three to create a fragmented heap.
4. Attempts to allocate one 1024-byte block; success proves that the coalescing logic merged the freed fragments correctly.

4 Process Manager

Primary contributors: Nazifa Fairuz Zuthi (Issue #3), Md. Redwan Bhuiyan Rafio (Issue #4).

4.1 Process Control Block (PCB)

Each process is represented by a `pcb_t` structure stored in a fixed-size global process table (`proctab[NPROC]`, `NPROC = 8`):

```

1 typedef struct pcb {
2     pid32    pid;           // Process ID
3     int      prstate;       // Current state
4     int      prprio;        // Dynamic priority
5     char     *prstkptr;     // Saved stack pointer
6     char     *prstkbase;    // Allocated stack base
7     int      next;          // Next slot in linked queue
8     // Scheduler fields

```



```

9  void (*prfunc)(void); // Entry-point function
10 int prquantum; // Assigned time quantum
11 int prtime; // Remaining quantum ticks
12 int original_prio; // Pre-aging priority
13 int prwait_time; // Ticks spent waiting
14 int prcputime; // Total CPU ticks consumed
15 // IPC
16 Message msg_inbox;
17 int has_msg;
18 pid32 sender_pid;
19 } pcb_t;

```

Listing 5: PCB definition (process.h)

4.2 Process States

Processes transition through six states:

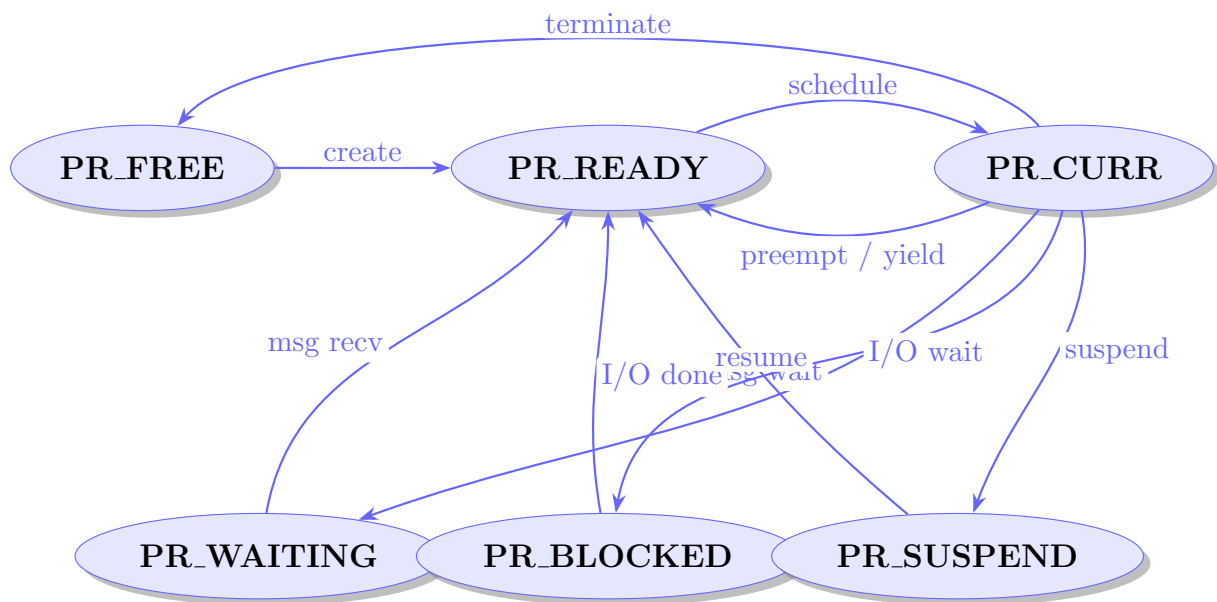


Figure 2: Process state machine. PR_BLOCKED, PR_WAITING, and PR_SUSPEND are defined for extensibility.

4.3 Process Creation

`create_process_with_func(priority, func)` performs the following steps atomically:

1. Scan proctab for a free slot (PR_FREE).
2. Allocate a `STACK_PER_PROC` byte stack via `heap_alloc()`.
3. **Fabricate a stack frame** compatible with `ctxsw`:
`[EBX=0] [ESI=0] [EDI=0] [EBP=0] [EIP=func]`. When the scheduler calls `ctxsw` for

the first time, it pops these registers and jumps to `func` as if returning from a normal call.

4. Initialise all PCB fields (PID, state, priority, quantum, IPC).
5. Enqueue the slot into the ready queue.
6. Return the new PID.

4.4 Ready Queue Implementation

The ready queue is a singly-linked list threaded through the `pcb_t.next` field (slot indices, not pointers). The `queue_t` structure holds the head and tail slot indices:

```

1 void q_insert(int slot, queue_t *q) {
2     proctab[slot].next = -1;
3     if (q->tail == -1) { q->head = q->tail = slot; }
4     else { proctab[q->tail].next = slot; q->tail = slot; }
5 }
6 int q_remove(queue_t *q) {
7     if (q->head == -1) return -1;
8     int slot = q->head;
9     q->head = proctab[slot].next;
10    if (q->head == -1) q->tail = -1;
11    proctab[slot].next = -1;
12    return slot;
13 }

```

Listing 6: Queue operations (process.c)

4.5 Utility Functions (Issue #4)

The following accessor functions are provided for safe, encapsulated access to PCB internals:

Function	Description
<code>getpid()</code>	Return current running PID
<code>find_slot(pid)</code>	Map PID to process table index
<code>get_process_state(pid)</code>	Return <code>prstate</code> value
<code>get_process_priority(pid)</code>	Return dynamic priority
<code>is_valid_pid(pid)</code>	Non-free process existence check
<code>get_stack_base(pid)</code>	Return <code>prstkbase</code> pointer
<code>get_num_ready()</code>	Count of processes in ready queue

5 Inter-Process Communication (IPC)

Contributor: Md. Redwan Bhuiyan Rafio (Issue #4 – bonus).

Each PCB embeds a single-slot **message inbox**:

```

1 #define MSG_SIZE 128
2 typedef struct message {
3     pid32 sender_pid;
4     char  data[MSG_SIZE];
5     int   len;
6 } Message;

```

Listing 7: Message structure

5.1 API

```
int send(pid32 dest, char *msg, int len)
```

Copies up to `MSG_SIZE` bytes into `dest`'s inbox and sets `has_msg = 1`. Returns 0 on success, `-1` on error.

```
int receive(pid32 src, char *buf, int max)
```

Reads from the current process's inbox if `has_msg` is set. If `src != -1`, it additionally verifies the sender. Clears the inbox on success. Returns bytes received or `-1`.

The current implementation is a single-slot, non-blocking mailbox: a new **send** overwrites an unread message. This is a deliberate simplification for the bare-metal context where no blocking I/O primitives are available.

6 CPU Scheduler

Contributor: Safius Sifat (Issue #5).

6.1 Scheduling Policies

Two scheduling policies are supported, selected at compile time via the global variable `sched_policy`:

Policy	Macro	Description
Round-Robin	<code>SCHED_RR</code>	FIFO order from ready queue
Priority Round-Robin	<code>SCHED_PRIO</code>	Highest-priority first; ties broken FIFO

The default policy is `SCHED_PRIO`.

6.2 Time Quantum

Each process has an independent, configurable time quantum:

Function	Description
<code>set_quantum(pid, q)</code>	Set quantum for a specific process
<code>get_quantum(pid)</code>	Query current quantum
<code>update_process_time()</code>	Decrement <code>prtime</code> , increment <code>prcputime</code>

The default quantum is `DEFAULT_QUANTUM = 10` ticks.

6.3 Context Switch

Context switching is performed by the handwritten x86 assembly routine `ctxsw` in `ctxsw.S`:

1. **Save** the current process's callee-saved registers (`EBX`, `ESI`, `EDI`, `EBP`) and the return address onto its stack; write `ESP` into `proctab[old].prstkptr`.
2. **Restore** the next process's `ESP` from `proctab[new].prstkptr`; pop its saved registers and return address, effectively resuming it.

The C interface is:

```
1 extern void ctxsw(void **old_sp, void **new_sp);
```

Listing 8: `ctxsw` declaration

6.4 Aging Anti-Starvation Mechanism (Bonus)

To prevent indefinite starvation of low-priority processes, the `apply_aging()` function is called on each `yield()`:

- Every `READY` process has its `prwait_time` incremented.
- When `prwait_time` reaches `AGING_THRESHOLD = 50`, the process's priority is boosted by `AGING_BOOST = 1` (up to a cap of 10), and the wait counter resets.
- When a process transitions to `PR_CURR`, its priority is restored to `original_prio` and its wait time resets.

6.5 Scheduler Data Flow

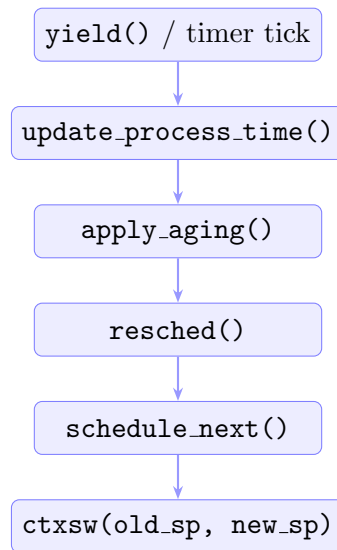


Figure 3: Scheduler execution path on each `yield()` call.

7 Serial I/O Driver

The serial driver (`serial.c` / `serial.h`) communicates over COM1 (base port 0x3F8) and provides:

- `serial_init()` – Configure baud rate, 8N1 framing, FIFOs.
- `serial_putc(c)` – Spin-wait until transmit buffer empty, then write one character.
- `serial_puts(str)` – Write a null-terminated string.
- `serial_getc()` – Spin-wait for incoming byte; used by the interactive shell.

Port I/O is performed through the inline helpers in `io.h`:

```

1 static inline void outb(uint16_t port, uint8_t val) {
2     __asm__ volatile("outb %0, %1" : : "a"(val), "Nd"(port));
3 }
4 static inline uint8_t inb(uint16_t port) {
5     uint8_t ret;
6     __asm__ volatile("inb %1, %0" : "=a"(ret) : "Nd"(port));
7     return ret;
8 }

```

Listing 9: Port I/O helpers (`io.h`)

8 Build System

The project is built with a single `Makefile` targeting a cross-compiled i686 ELF binary:

```
make          # build kacchiOS.bin
make run      # launch QEMU (qemu-system-i386)
make clean    # remove build artefacts
```

Key compiler flags:

Flag	Purpose
-m32	Target 32-bit i686
-ffreestanding	No host libc/start files
-nostdlib	Link without standard libraries
-O2	Optimisation
-Wall -Wextra	Enable warnings

The linker script `link.ld` loads the kernel at `0x00100000` (1 MB physical), places `.text`, `.rodata`, `.data`, then `.bss`, and exports `__bss_start` / `__bss_end` symbols consumed by `boot.S` to zero-initialise the BSS segment.

9 Testing

All tests are run automatically at startup in `kmain()` before the interactive shell activates. Results are printed over the serial port.

9.1 Memory Tests

- Stack alloc / free cycle.
- Best-fit selection across multiple allocation sizes.
- Fragmentation and coalescing: allocate 3×512 B, free all, then allocate 1024 B – must succeed.

9.2 Process Manager Tests (10 tests)

#	Test	Verifies
1	Process Creation	Three PIDs returned, all $\neq -1$
2	State Check	All newly created processes in <code>PR_READY</code>
3	Priority Check	<code>get_process_priority()</code> returns correct value
4	Ready Queue Count	<code>get_num_ready() == 3</code>
5	Valid PID	<code>is_valid_pid()</code> true for all created PIDs
6	State Transition	After <code>set_current()</code> , state is <code>PR_CURR</code>
7	Queue After Curr	<code>get_num_ready() == 2</code> after one runs
8	Termination	<code>terminate_process()</code> returns 0; PID invalidated
9	Queue After Term	Queue count remains correct
10	Stack Verification	Live process has stack base; terminated process has NULL

9.3 IPC Tests (7 tests)

#	Test	Verifies
IPC-1	Process Creation	Sender and receiver processes created successfully
IPC-2	Send Message	<code>send()</code> returns 0
IPC-3	Message Available	<code>has_msg == 1</code> in receiver's PCB
IPC-4	Sender ID	<code>msg_inbox.sender_pid == sender</code>
IPC-5	Message Length	<code>msg_inbox.len == 10</code>
IPC-6	Message Content	All 10 bytes match "Hello IPC!"
IPC-7	Receive with Check	<code>receive(src, buf, max)</code> returns 10

9.4 Scheduler Tests (5 tests)

#	Test	Verifies
SCHED-1	Process w/ Function	<code>create_process_with_func()</code> works for all 3
SCHED-2	Function Pointers	PCB <code>prfunc</code> fields correctly set
SCHED-3	Default Quantum	All processes have quantum = 10
SCHED-4	Custom Quantum	<code>set_quantum(p1, 20)</code> reflects in <code>get_quantum()</code>
SCHED-5	Priority Selection	Highest-priority process selected by <code>schedule_next()</code>

10 Team Contributions

Member	GitHub	Issue		Responsibilities
Md. Sajedul Islam	sajidruetcse21	#1	Memory (Core)	Memory layout design, <code>stack_alloc/stack_free</code> , initial heap allocator (first-fit → best-fit), free-block data structures.
Tanmoy Mridha	Tanmoym047	#2	Memory (Advanced)	Stack deallocation edge cases, <code>heap_free</code> with forward merge and global coalesce, fragmentation handling, full stress test suite.
Nazifa Fairuz Zuthi	ZohanaZuthi	#3	Process (Core)	PCB design, process table (<code>proctab</code>), <code>create_process</code> , <code>terminate_process</code> , basic state machine (READY/CURR/TERMINATED).
Md. Redwan Bhuiyan Rafio	rafio1020	#4	Process (Utilities)	Utility accessors (<code>getpid</code> , <code>find_slot</code> , etc.), extended states (BLOCKED/WAITING/SUSPEND), IPC message-passing subsystem (<code>send/receive</code>).
Safius Sifat	Safius-Sifat	#5	Scheduler	Scheduling policies (RR & priority RR), <code>resched/yield</code> , <code>ctxsw.S</code> context switch, configurable quantum, aging anti-starvation bonus.
Shah Sanjida Rahman Nitu	Sanjida-Rahman-Nitu	#6	Docs	Led documentation and finalization: authored <code>README.md</code> , compiled this project report, prepared the test checklist, and directed and edited the demo video (<code>Demo_Video.mp4</code>).

11 Challenges and Design Decisions

No standard library. All string operations, type definitions, and I/O abstractions had to be implemented from scratch (`string.c`, `types.h`, `serial.c`).

Stack-frame fabrication. Getting a newly created process to resume correctly via `ctxsw` required carefully laying out the initial stack frame so that the callee-saved register pops lead to a clean entry into the process function.

Best-fit vs. first-fit. The initial implementation used first-fit. After stress testing revealed early fragmentation, the allocator was upgraded to best-fit combined with block splitting and global coalescing.

IPC mailbox limitation. A true blocking receive requires sleep/wakeup primitives tied to a timer interrupt. Since kacchiOS does not yet configure the PIC or handle hardware interrupts, the IPC mailbox is deliberately non-blocking: the sender overwrites any unread message. Blocking IPC is listed as future work.

Quantum-based scheduling without a hardware timer. Because no timer interrupt is wired up yet, time quantum tracking is cooperative: processes call `yield()` explicitly. The quantum counter is decremented on each `yield()` call, which still demonstrates the scheduling mechanism correctly.

12 Conclusion

kacchiOS successfully demonstrates the foundational layers of an operating system kernel running directly on bare-metal x86 (or QEMU). Starting from a Multiboot-compatible assembly stub, the project implements:

- A dual-strategy memory manager (bump-pointer stack + best-fit heap with fragmentation-aware coalescing).
- A full process lifecycle with a six-state machine, a PCB-based process table, and a thread-safe linked ready queue.
- A context-switching scheduler supporting both Round-Robin and Priority Round-Robin policies, configurable per-process time quanta, and an aging mechanism to prevent starvation.
- A lightweight non-blocking IPC mailbox.
- 22 automated integration tests spanning memory, process, IPC, and scheduling subsystems.

All 22 tests pass on QEMU 7.x, confirming that the modules integrate correctly. The

codebase is modular, well-documented, and provides a solid foundation for the future enhancements listed above.